

Assignment 1

Architectural Design and Trade-Off Analysis of a Real-Time Ride-Sharing Platform Backend

Due Date: Check Moodle for confirmation.

Submission Format: A written DOCX or PDF report (7–10 pages). Diagrams and architectural illustrations are encouraged.

Demonstration: A demo is not required unless specifically requested by the instructor or the TA.

Team Size: This assignment may be completed individually or in groups of up to three (3) students. If you choose to work in a team, please include the names of all team members on the cover page of your report and ensure that the assignment is submitted through Moodle. **Important:** Once the assignment has been submitted, no changes to the group membership will be permitted. Please ensure that your team composition is finalized before submitting.

Important Note: Some of the requirements in this assignment intentionally involve concepts, technologies, or implementation details that may not have been covered explicitly during lectures. This is by design. In distributed systems, engineers are frequently required to investigate unfamiliar topics, consult technical documentation, evaluate different approaches, and independently acquire the knowledge needed to solve complex problems. You are therefore expected to conduct additional research using appropriate resources such as academic papers, official documentation, technical articles, and other credible references. Your ability to explore these topics, develop a solid understanding of them, and apply that knowledge effectively is an important part of the learning objectives of this assignment. Be sure to justify your design choices and demonstrate the insights you gained through your research.

Assignment Overview: In this assignment, you are required to architect the backend infrastructure of a **real-time ride-sharing platform** similar to Uber or Lyft, using principles from distributed systems. Your task is to **analyze and design a distributed backend system** that supports high availability, low latency, and data consistency under high concurrency.

You must integrate and analyze the following core architectural components:

- **Database Sharding** (Hash-based, Range-based, or Consistent Hashing)
 - **Replication and Consistency Models** (Eventual, Strong, Quorum-based)
 - **Load Balancing** (Layer 4 vs. Layer 7)
-

Use Case Scenario: You are designing the backend system for "**SwiftRide**", a real-time urban ride-sharing service.

System Functional Requirements:

- Drivers broadcast live location and availability.
- Riders send trip requests that are matched with nearby drivers.
- The system supports live matching, route calculation, and payment processing.
- Data includes user accounts, trip histories, real-time GPS data, and pricing rules.

Non-Functional Requirements:

- Global scale support (multi-region architecture assumed)
 - <200ms latency for ride-matching response in 90% of requests
 - High read/write throughput for hotspots (e.g., downtown at 5 PM)
-

Your Design Tasks:

1. Architecture Design

- Propose a high-level architectural model that integrates:
 - Distributed data storage (with sharding strategy and justification)
 - Replication for availability
 - Load balancing layer with smart routing logic (L4 or L7)

2. Sharding Strategy Trade-Off Analysis

- Compare at least two sharding strategies (e.g., hash vs. range or consistent hashing)
- Discuss which parts of the dataset (e.g., user location, trip records) benefit from each approach
- Analyze range queries (e.g., find all drivers in a 2km radius) and their impact on data partitioning

3. Consistency Model Justification

- Propose a consistency model (strong/eventual/quorum) for different services:
 - Driver location updates
 - Rider-trip assignment
 - Payment transactions
- Justify each choice based on real-world trade-offs (e.g., CAP theorem)

4. Design Diagrams (Required)

- At minimum, include:
 - System architecture diagram
 - Data flow/sequence diagram for ride-matching

Evaluation Criteria:

1. Clarity and justification of architectural decisions
 2. Depth of analysis on trade-offs and alternatives
 3. Logical consistency and completeness of design
 4. Quality and clarity of visuals and document structure
-

✓ Grading Rubric

Category	Criteria	Points
1. Architecture Design	- Clear overall architecture integrating sharding, replication, consistency, and load balancing - Logical modular decomposition and interaction between services - Use of appropriate technologies (e.g., database type, load balancer layer)	25%
2. Sharding Strategy Trade-Off Analysis	- Compares at least two sharding strategies - Clear justification of chosen strategy for different datasets - Discussion on range queries and partitioning impact	20%
3. Consistency Model Justification	- Proposes and differentiates consistency models per service (location, assignment, payment) - Accurate reference to CAP theorem trade-offs - Well-reasoned justifications	15%

4. Use of Diagrams	- System architecture diagram is complete and easy to interpret - Includes sequence diagram for ride-matching - Diagrams are labeled, relevant, and contribute to clarity	15%
5. Depth of Analysis & Justification	- Design choices are supported with sound reasoning - Alternatives considered and trade-offs discussed - Alignment with distributed systems principles	15%
6. Presentation & Structure	- Organized report with clear section headings - 5–7 pages, free of grammar/spelling errors - Consistent formatting and citation of any external references	10%